**República Bolivariana de Venezuela**

**Ministerio del Poder Popular para la Educación Universitaria**

**Universidad Politécnica Territorial José Antonio Anzoátegui**

**Extensión - Guanta**




# INFORME DE TEORÍA: PROGRAMACIÓN II

**Profesor:**

**Humner Lenis**

**Alumno:**

**Franco Alvarado**

**Guanta 09/11/2026**

# El Paradigma Orientado a Objetos (POO)

La Programación Orientada a Objetos es un paradigma de programación que utiliza "objetos" y sus interacciones para diseñar aplicaciones y programas informáticos. A diferencia de la programación estructurada tradicional (que se centra en funciones y procedimientos lineales), la POO se basa en modelar el mundo real o los requerimientos del sistema agrupando datos y el comportamiento que opera sobre esos datos en una sola entidad.

Los pilares fundamentales que sustentan este paradigma son:

**Abstracción:** Permite aislar los elementos esenciales de un objeto, ignorando los detalles irrelevantes o complejos para centrarse en lo que hace el objeto dentro del sistema.

**Encencapsulamiento:** Mecanismo que agrupa los datos (atributos) y los métodos (funciones) que operan sobre esos datos en una sola unidad, restringiendo el acceso directo a algunos componentes para proteger la integridad de la información.

**Herencia:** Capacidad de crear nuevas clases basadas en clases existentes, permitiendo reutilizar código y establecer jerarquías lógicas.

**Polimorfismo:** Propiedad mediante la cual se puede enviar un mismo mensaje a objetos de diferentes clases, logrando que cada objeto responda de manera distinta según su propia implementación.

# UML (Lenguaje Unificado de Modelado) y la Arquitectura de Software

Antes de escribir código, es indispensable representar gráficamente la arquitectura del sistema para visualizar, especificar y documentar los componentes de un software orientado a objetos.

**Diagramas de Clases:** Son los diagramas estructurales más importantes en UML. Permiten modelar estáticamente el sistema mostrando las clases que lo componen, sus atributos, sus operaciones (métodos) y las relaciones estáticas entre ellas (como asociaciones, dependencias, herencia y agregación).

**Representación Gráfica:** En un diagrama UML, una clase se representa típicamente mediante un rectángulo dividido en tres compartimentos:

1. **Nombre de la clase** (en la parte superior, usualmente en negrita).
2. **Atributos o propiedades** (en el compartimento central, indicando su visibilidad: + público, - privado, # protegido).

3. **Métodos u operaciones** (en el compartimento inferior, detallando las acciones que puede realizar la clase).

## Clases y Objetos

En el corazón de la POO se encuentran estos dos conceptos interdependientes:

**La Clase (La Plantilla):** Es una definición abstracta, un plano o molde que define qué atributos y qué comportamientos (métodos) tendrán los objetos que pertenezcan a esa clase. Por sí misma, una clase no almacena datos particulares, sino la estructura general.

**El Objeto (La Instancia):** Es una instancia concreta de una clase. Cuando se crea un objeto a partir de una plantilla (clase), este adquiere su propia identidad y un espacio en memoria con valores específicos para sus atributos.

**Comparativa conceptual en lenguajes modernos:**

**En Python:** Se define una clase utilizando la palabra reservada class y se instancia llamando al nombre de la clase como una función (p1 = Persona()).

**En PHP:** De manera similar, se estructura con class Persona {} y se instancia explícitamente utilizando el operador de asignación de memoria y la palabra clave new ($p1 = new Persona();).

## Especificadores de Visibilidad (Control de Visibilidad)

Los especificadores de visibilidad (o modificadores de acceso) son palabras clave o convenciones que determinan el nivel de acceso y la encapsulación de los atributos y métodos dentro de una clase, protegiendo los datos de alteraciones no autorizadas desde el exterior.

Los niveles de visibilidad estándar son:

**Público (**public**):** Los miembros declarados como públicos son accesibles desde cualquier lugar del programa: dentro de la misma clase, desde clases heredadas y desde fuera de la clase (instancias).

**Privado (**private**):** Restringen el acceso exclusivamente al interior de la clase donde fueron declarados. Ninguna otra clase (ni siquiera las hijas) puede acceder a ellos directamente.

**Protegido (**protected**):** Permiten el acceso dentro de la propia clase y a través de las clases hijas (herencia), pero se mantienen ocultos para los objetos externos o clases no relacionadas.

**Diferencias sintácticas entre lenguajes:**

**En Python:** Se simulan comúnmente mediante convenciones. Los atributos protegidos se anteponen con un guion bajo único (_atributo), mientras que los privados utilizan doble guion bajo (__atributo), activando un mecanismo de ocultamiento de nombres (*name mangling*).

**En PHP:** Se implementan de forma explícita mediante las palabras reservadas del lenguaje: public, protected y private antes de declarar la propiedad o el método.

## Constructores (Método de Inicialización)

Un constructor es un método especial de una clase que se ejecuta **automáticamente** al momento de instanciar un objeto (es decir, cuando se crea usando la palabra new o llamando a la clase). Su función principal es asignar valores iniciales a los atributos del objeto o ejecutar configuraciones previas necesarias para su funcionamiento.

**En Python:** Se define obligatoriamente con el nombre especial __init__(self, ...), donde self representa la instancia actual del objeto.

**En PHP:** Se define utilizando la sintaxis estándar de métodos precedida por la palabra clave __construct(...).

## Herencia y Reutilización de Código

La herencia es uno de los pilares más potentes de la POO. Permite definir una nueva clase (**Clase Hija o Subclase**) basada en una clase ya existente (**Clase Padre o Superclase**).

**Propósito:** Evita la duplicación de código (*principio DRY - Don't Repeat Yourself*). La clase hija hereda automáticamente todos los atributos y métodos públicos y protegidos de la clase padre, pudiendo además añadir sus propios atributos específicos o sobrescribir comportamientos existentes.

**Sintaxis comparativa:**

**En Python:** Se indica pasando la clase padre entre paréntesis al declarar la hija: class Alumno(Persona):.

**En PHP:** Se utiliza la palabra reservada extends: class Alumno extends Persona.

Aquí tienes la **Tercera Parte** de la teoría para tu informe de Programación II, abarcando Clases Abstractas y Polimorfismo según el plan de evaluación.

## Clases Abstractas

Una clase abstracta es una clase diseñada específicamente para servir como **plantilla base o superclase** en una jerarquía de herencia, con una característica fundamental: **no se pueden instanciar directamente** (es decir, no se pueden crear objetos usando un constructor directo de esa clase).

**Propósito:** Agrupar atributos y comportamientos comunes que serán compartidos por múltiples clases hijas, obligando a estas últimas a implementar ciertos métodos específicos. Aseguran que las subclases sigan una estructura estandarizada.

**Métodos Abstractos:** Son métodos declarados en la clase abstracta que no contienen cuerpo o código de ejecución; su única función es definir un "contrato" que obliga a todas las clases hijas a desarrollarlos obligatoriamente.

**Implementación sintáctica:**

**En Python:** Se gestionan importando el módulo especializado abc (*Abstract Base Classes*), heredando de la clase base ABC y utilizando el decorador @abstractmethod.

**En PHP:** Se declaran explícitamente anteponiendo la palabra reservada abstract tanto a la clase (abstract class) como a los métodos abstractos que contendrá.

## Polimorfismo

El polimorfismo (que se traduce literalmente como "muchas formas") es uno de los principios más avanzados de la POO. Describe la capacidad que tienen diferentes objetos de responder a una **misma llamada de método**, pero ejecutando un comportamiento interno totalmente distinto y adaptado a la naturaleza de cada clase particular.

**Sobrescritura de Métodos (*Method Overriding*):** Es el mecanismo clave que hace posible el polimorfismo. Ocurre cuando una clase hija redefine un método que ya había heredado de su clase padre. Aunque el nombre del método y sus parámetros se mantengan idénticos, la lógica interna de ejecución cambia para ajustarse a las necesidades de la subclase.

## Comparativa de Aplicación (Python vs. PHP)

**En Python:**

```
from abc import ABC, abstractmethod
```

```
class Figura(ABC):

  @abstractmethod
  def calcular_area(self):
    pass


class Cuadrado(Figura):

  def calcular_area(self):
    # Sobrescribe el método abstracto de la clase padre
    return self.lado * self.lado
```

**En PHP:**

```
abstract class Figura {
    abstract public function calcular_area();
}

class Cuadrado extends Figura {
    public function calcular_area() {
        // Sobrescribe el método abstracto de la clase
padre
        return $this->lado * $this->lado;
    }
}
```

## Interfaces

Una interfaz funciona como un **contrato estricto de métodos obligatorios** que define qué acciones debe realizar una clase, pero sin especificar cómo debe hacerlo (no contiene lógica ni código interno en sus métodos). Cualquier clase que implemente una interfaz está obligada a desarrollar todos y cada uno de los métodos declarados en ella.

**Propósito:** Garantizar que diferentes clases compartan un comportamiento común y estandarizado, permitiendo que el sistema interactúe con ellas de manera uniforme sin importar sus diferencias internas.

**Implementación sintáctica:**

**En Python:** Se suelen modelar mediante clases base abstractas o protocolos usando la estructura del módulo abc (class Protocolo (ABC):).

**En PHP:** Se implementan de forma nativa y directa utilizando la palabra reservada interface.

## Arquitectura Web (Patrones MVC y MVT)

En el desarrollo de aplicaciones web modernas, la arquitectura de software se basa en separar las responsabilidades del sistema para facilitar el mantenimiento, la escalabilidad y el trabajo en equipo. Los frameworks backend implementan patrones arquitectónicos específicos:

**MVC (Model - View - Controller):** Usado habitualmente en frameworks basados en PHP como **Laravel**.

**Modelo (Model):** Gestiona la capa de datos, la lógica de la base de datos y las reglas de negocio.

**Vista (View):** Representa la interfaz de presentación visual que recibe el usuario.

**Controlador (Controller):** Actúa como intermediario; recibe las peticiones del usuario, se comunica con el modelo para procesar los datos y selecciona la vista adecuada para responder.

**MVT (Model - View - Template):** Usado habitualmente en frameworks basados en Python como **Django**.

**Modelo (Model):** Define la estructura de los datos y la interacción con la base de datos.

**Vista (View):** Contiene la lógica que procesa la solicitud web, recupera datos del modelo y decide qué renderizar (en Django, la "Vista" actúa conceptualmente de manera similar al Controlador de MVC).

**Template (Plantilla):** Es la capa visual y de diseño que combina HTML y variables de datos para mostrárselas al usuario final.

## 3. Diseño (HTML/CSS) y Motores de Plantillas

La interfaz visual de una aplicación web requiere de tecnologías de marcado y estilizado como **HTML** (para estructurar el contenido) y **CSS** (para la presentación visual y el diseño estético). Los frameworks backend integran sistemas de plantillas para dinamizar estas vistas:

**En Python / Django:** Las interfaces visuales se estructuran dentro del directorio templates/, haciendo uso de motores de renderizado avanzados y seguros

como **Jinja2**, que permiten inyectar variables de datos y estructuras de control (como bucles y condiciones) directamente en el HTML.

**En PHP / Laravel:** Las vistas se organizan en la ruta resources/views/ empleando el motor de plantillas **Blade**, el cual optimiza la escritura de código HTML mediante sintaxis limpia, herencia de plantillas y componentes reutilizables.